

SICP Portfolio Projects — Chapter-wise (Scheme, First Pass)

Each project lists its **Purpose** (what it teaches), **Scope** (the first-pass build), and **Extensions** (where to stretch it later). Section numbers refer to *Structure and Interpretation of Computer Programs*, 2nd ed.

Contents

1. [Building Abstractions with Procedures](#)
2. [Building Abstractions with Data](#)
3. [Modularity, Objects, and State](#)
4. [Metalinguistic Abstraction](#)
5. [Computing with Register Machines](#)

Chapter 1 — Building Abstractions with Procedures

Numerical methods toolkit

- **Purpose:** first-class procedures, recursion vs iteration, fixed points, functions as arguments/return values.
- **Scope:** half-interval + Newton root finders (§1.3.3–1.3.4), `sum` / `product` / `accumulate` (§1.3.1), Simpson integration (Ex 1.29), `average-damp`, `compose` / `repeated` (Ex 1.42–1.43), numerical `deriv`.
- **Extensions:** continued fractions for $e/\tan$ (Ex 1.37–1.39), series accelerators, a "function-as-data" calculus lib, later run it over Ch2's generic numbers.

Number-theory / primality library

- **Purpose:** orders of growth, fast exponentiation, probabilistic algorithms.
- **Scope:** trial division, fast `expt` (§1.2.4), Fermat test (§1.2.6), Miller–Rabin (Ex 1.28), GCD.
- **Extensions:** Carmichael hunter (Ex 1.27), a timing harness comparing $O(\sqrt{n})$ vs $O(\log n)$, toy RSA once you add modular inverse.

Chapter 2 — Building Abstractions with Data

Symbolic differentiator

- **Purpose:** symbolic manipulation, data abstraction, representation choice.
- **Scope:** sums/products/exponents with constructor-level simplification (§2.3.2).
- **Extensions:** quotients, chain rule (trig/exp/log), a like-terms simplifier, infix parser, pretty-printer.

Generic arithmetic system (type tower)

- **Purpose:** data-directed programming, tagged data, dispatch, coercion (§2.4–2.5).
- **Scope:** integer/rational/real/complex (rectangular + polar), generic `add/sub/mul/div`, installable packages.
- **Extensions:** `raise / project / drop` coercion tower (Ex 2.83–2.85), bignums, modular-arithmetic type, vectors/matrices as types.

Polynomial algebra (on top of the tower)

- **Purpose:** recursive/hierarchical data, generic ops over generic coefficients.
- **Scope:** univariate polynomials, add/mul over the tower (§2.5.3).
- **Extensions:** polynomial division + GCD (Ex 2.94), rational functions, multivariate, both sparse and dense representations.

Huffman encoder/decoder

- **Purpose:** trees, sets, a real algorithm end-to-end.
- **Scope:** build tree from frequencies, encode/decode (§2.3.4).
- **Extensions:** compress a real file + report ratio, adaptive Huffman, compare vs fixed-length encoding.

Picture language (Escher)

- **Purpose:** closure of combination means, higher-order ops on data — the payoff chapter for "abstraction as power." MIT Scheme ships the graphics.
 - **Scope:** painters, `beside` / `below` / `flip` , `square-limit` (§2.2.4).
 - **Extensions:** new base painters, deeper recursion schemes, SVG/PostScript export, parameterized fractals.
-

Chapter 3 — Modularity, Objects, and State

Digital circuit simulator

- **Purpose:** event-driven simulation, mutable state, the agenda, message-passing modularity.

- **Scope:** wires, and/or/inverter gates with delays, agenda queue, half/full/ripple-carry adder (§3.3.4).
- **Extensions:** xor/mux, a circuit-description parser, waveform output, build an ALU.

Constraint propagation system

- **Purpose:** bidirectional/nondirectional computation, connectors.
- **Scope:** connectors + adder/multiplier/constant constraints, the C/F converter (§3.3.5).
- **Extensions:** squaring/exp constraints, a DSL for declaring networks, contradiction detection, a mini dependency-spreadsheet.

Streams + power-series engine

- **Purpose:** delayed evaluation, infinite data, decoupling order from time (§3.5).
- **Scope:** `cons-stream` + stream ops, prime sieve, stream integration, series for exp/sin/cos (§3.5.2–3.5.3).
- **Extensions:** lazy series arithmetic reusing Ch2's generic ops, ODE solving via streams (§3.5.4), random streams + Monte Carlo (§3.5.5).

Account / concurrency simulation (lower priority)

- **Purpose:** assignment, the environment model, serializers, the cost of shared state (§3.4).
 - **Scope:** `make-account` , mutators, `make-serializer` (§3.4.2).
 - **Extensions:** induce and then fix a deadlock, model a small transaction system.
-

Chapter 4 — Metalinguistic Abstraction

Metacircular evaluator (the flagship)

- **Purpose:** eval/apply, environments, how interpreters work (§4.1).
- **Scope:** self-eval, variables, `quote/if/lambda/define/set!/begin/cond`, application, environments, REPL.
- **Extensions:** `let/let*/letrec/named-let`, internal-definition scanning (§4.1.6), the analyzing evaluator for speed (§4.1.7), tail-call optimization, embed your Ch2 algebra as primitives.

Lazy evaluator

- **Purpose:** normal-order evaluation, thunks, memoization (§4.2).

- **Scope:** delay arguments in the metacircular evaluator; lazy lists.
- **Extensions:** mixed lazy/strict via `lazy` / `lazy-memo` params (Ex 4.31), streams with no special forms, lazy pairs.

Nondeterministic evaluator (`amb`)

- **Purpose:** automatic backtracking, control as continuations, generate-and-test (§4.3).
- **Scope:** `amb` / `require`, solve multiple-dwelling and Pythagorean-triples puzzles.
- **Extensions:** N-queens / map-coloring / Sudoku, a small natural-language parser (§4.3.2), `if-fail`, `ramb`.

Logic-programming query engine (mini-Prolog)

- **Purpose:** declarative programming, pattern matching, unification, streams of frames (§4.4).
 - **Scope:** assertions + rules, `and` / `or` / `not` / `lisp-value`, the microshaft database.
 - **Extensions:** full unification (§4.4.2, Ex 4.61+), load facts from a file, a proper REPL, demos like family trees, graph reachability, or toy type inference.
-

Chapter 5 — Computing with Register Machines

Register-machine simulator

- **Purpose:** how machines execute — registers, controller, sequencing, the stack (§5.1–5.2).
- **Scope:** `make-machine`, registers/stack/controller, run GCD, recursive factorial, Fibonacci.
- **Extensions:** instruction tracing + profiling, breakpoints and stack monitoring (§5.2.4), a cleaner assembler/DSL.

Storage allocation + garbage collector

- **Purpose:** the memory model, pairs-as-vectors, stop-and-copy GC (§5.3).
- **Scope:** vector-backed memory, allocation, stop-and-copy collector.
- **Extensions:** mark-sweep variant, compaction/GC stats, generational ideas.

Explicit-control evaluator (bridges Ch4 ↔ Ch5)

- **Purpose:** a real interpreter expressed as a register machine; proper tail recursion falls out (§5.4).
- **Scope:** the ECEval controller; run your Ch4 language on the simulator.

- **Extensions:** add the special forms you built in Ch4, in-machine error handling, performance monitoring.

Compiler: Scheme → register code (the hardest synthesis in the book)

- **Purpose:** compilation, code generation, linkage, compile-time environments (§5.5).
- **Scope:** compile expressions to instruction sequences, run them on the simulator.
- **Extensions:** lexical addressing (§5.5.6), open-coding primitives (§5.5.7), peephole optimization, compiled-vs-interpreted benchmarks.